

EK-A: Kodlar ve Algoritmalar

Tez Raporu Eki

Orhun Uzdiyem | Aralık 2024

Biyo-yağ buhar reforming süreci için geliştirilen Ters Makine Öğrenmesi modellerinin kaynak kodları ve algoritmaları.

1. Proje Genel Bakış ve Dizin Yapısı

Proje Amacı

Ters makine öğrenmesi yaklaşımı kullanarak:

- **Giriş:** Singaz kompozisyonu (H₂, CO, vs.) + Proses koşulları (T, P, S/C)
- **Çıkış:** Biyo-yağ kompozisyonu (6 ana bileşen)

Bu ek belge, bu tahmin modellerinin geliştirilmesi için kullanılan tüm yazılım altyapısını sunmaktadır.

```
reverse_ml_biooil_to_product/ |—— cantera_generation/ # Simülasyon
sistemi | |—— generate_data.py # Ana kontrolcü | |—— modules/ #
Denge & Ayırma |—— ml_models/ # Temel ML modülleri |——
ml_reverse_prediction/ # Ana ML pipeline | |—— src/ | | |——
data_loader.py | | |—— deep_learning.py | | |—— ensemble_models.py
| |—— data/ # İşlenmiş veriler | |—— models/ # Eğitilmiş .pkl/.h5
|—— optimization_control/ # MPC & Optimizasyon
```


2.1 Veri Oluřturma Stratejisi (Cantera)

Simölasyon Matrisi

Toplam **1,350** veri noktası:

- 30 farklı biyo-yağ
- 5 Sıcaklık (650-850°C)
- 3 Basınç (5-30 bar)
- 3 S/C Oranı (2.0-6.0)

Vekil Moleküller

Termodinamik modelleme için:

- Aromatikler → Toluen
- Asitler → Asetik asit
- Alkoller → Etanol
- Furanlar → Furan

Süreç Akışı

1. Reformer Dengesi
2. WGS (HTS & LTS)
3. Flash Ayırma
4. PSA Saflařtırma

2.2 Ana Kontrolcü Kodu (generate_data_cantera.py)

Bu script tüm iş akışını yönetir:

1. Simülasyon matrisini yükler (1,350 senaryo)
2. Her senaryo için `run\_single\_simulation` fonksiyonunu çağırır.
3. Denge, WGS ve ayırma süreçlerini sırayla yürütür.
4. Sonuçları doğrular ve SQL veritabanına yazar.

```
class CanteraDataGenerator:
    def run_single_simulation(self, cantera_input):
        # Aşama 1: Reformer dengesi
        reformer_out = self.equilibrium.reformer_equilibrium(
            composition, T_K, P_Pa
        )

        # Aşama 2-3: WGS Reaksiyonları
        hts_out = self.equilibrium.wgs_equilibrium(reformer_out, ...)
        lts_out = self.equilibrium.wgs_equilibrium(hts_out, ...)

        # Aşama 4-6: Ayırma Prosesleri
        flash_out = self.separation.flash_separation(lts_out)
        h2_product = self.separation.psa_separation(flash_out)

        return simulation_data
```


2.3 Denge Hesaplayıcı (cantera_equilibrium.py)

Gibbs serbest enerji minimizasyonu kullanarak kimyasal dengeyi hesaplar.

- **GRI-Mech 3.0** mekanizmasını kullanır.
- Newton-Raphson yöntemi ile Gibbs enerjisini minimize eder.
- Entalpi, entropi ve yoğunluk gibi özellikleri döndürür.

```
class EquilibriumCalculator:
    def __init__(self, mechanism='gri30.yaml'):
        self.gas = ct.Solution(mechanism)

    def reformer_equilibrium(self, composition, T_K, P_Pa):
        # Başlangıç durumunu ayarla
        self.gas.TPX = T_K, P_Pa, composition

        # Gibbs minimizasyonu (TP sabit)
        self.gas.equilibrate('TP')

        return {
            'mole_fractions': self.gas.X,
            'enthalpy': self.gas.enthalpy_mole,
            'temperature_K': self.gas.T
        }
```


3.1 Veri Yükleme (data_loader.py)

Veritabanından simülasyon sonuçlarını çeker ve temizler.

- **Girişler:** Proses koşulları + Singaz kompozisyonu.
- **Hedefler:** 6 adet biyo-yağ bileşeni yüzdesi.
- Eksik verileri temizler ve kompozisyon toplamalarını doğrular.

```
def load_from_database(self):  
    query = """  
    SELECT  
        s.Temperature_C, s.Pressure_bar, s.SC_Ratio,  
        o.H2_molpercent, o.CO_molpercent, o.CO2_molpercent, ...  
        b.aromatics, b.acids, b.alcohols, b.furans ...  
    FROM ReformerSimulation s  
    INNER JOIN Biooil b ON s.BiooilID = b.BiooilId  
    WHERE s.ConvergenceStatus = 'Converged'  
    """  
  
    df = pd.read_sql(query, self.conn)  
    return df
```


3.2 Veri Ön İşleme ve Bölme

Veri Bölme Stratejisi

Toplam 1,350 örnek şu şekilde bölünmüştür:

Eğitim (%70): 944 örnek

Doğrulama (%15): 203 örnek

Test (%15): 203 örnek

</> Kod Parçacığı

```
def split_data(df, test_size=0.15, val_size=0.15):  
    X = df[self.input_features]  
    y = df[self.target_features]  
  
    # Önce test setini ayır  
    X_temp, X_test, y_temp, y_test = train_test_split(  
        X, y, test_size=test_size  
    )  
  
    # Kalanı eğitim ve doğrulama olarak böl  
    val_adj = val_size / (1 - test_size)  
    X_train, X_val, y_train, y_val = train_test_split(  
        X_temp, y_temp, test_size=val_adj  
    )  
    return X_train, X_val, X_test, ...
```


4.1 Random Forest Modeli

Temel (Baseline) model olarak Random Forest Regressor kullanılmıştır.

Hiperparametreler:

- 100 ağaç (n_estimators)
- Maksimum derinlik: 20
- Min samples split: 5

```
def train_random_forest(self, X_train, y_train):  
    models = {}  
    for component in self.target_features:  
        model = RandomForestRegressor(  
            n_estimators=100,  
            max_depth=20,  
            min_samples_split=5,  
            n_jobs=-1  
        )  
        model.fit(X_train, y_train[component])  
        models[component] = model  
  
    return models
```


4.2 XGBoost Modeli

Gradient Boosting algoritması ile daha güçlü bir baseline hedeflenmiştir.

Özellikler:

- Early stopping ile overfitting önleme.
- Düşük öğrenme oranı (0.05).
- %80 alt örnekleme (subsample).

```
def train_xgboost(self, X_train, y_train, X_val, y_val):  
    model = XGBRegressor(  
        n_estimators=200,  
        max_depth=10,  
        learning_rate=0.05,  
        subsample=0.8,  
        colsample_bytree=0.8  
    )  
  
    # Early stopping ile eđit  
    model.fit(  
        X_train, y_train[component],  
        eval_set=[(X_val, y_val[component])],  
        early_stopping_rounds=10  
    )
```


5.1 Derin Öğrenme Mimarisi (MLP)

Çok Katmanlı Algılayıcı (MLP) mimarisi:

- **Giriş (8):** Proses + Singaz
- **Gizli 1 (128):** ReLU + BN + Dropout(0.3)
- **Gizli 2 (64):** ReLU + BN + Dropout(0.2)
- **Gizli 3 (32):** ReLU + Dropout(0.1)
- **Çıkış (6):** Biyo-yağ bileşenleri

```
def build_mlp_model(input_dim, output_dim):  
    model = keras.Sequential()  
  
    # Katman 1: 128 Nöron  
    model.add(layers.Dense(128, activation='relu'))  
    model.add(layers.BatchNormalization())  
    model.add(layers.Dropout(0.3))  
  
    # Katman 2: 64 Nöron  
    model.add(layers.Dense(64, activation='relu'))  
    ...  
  
    # Çıkış Katmanı  
    model.add(layers.Dense(output_dim, activation='linear'))  
    return model
```


5.2 MLP Eğitimi ve Değerlendirme

Eğitim sürecinde stabilite için Callbacks kullanılmıştır:

- **EarlyStopping:** 30 epoch sabır.
- **ReduceLROnPlateau:** Öğrenme oranını dinamik azaltma.
- **Normalizasyon:** StandardScaler (z-score) kullanılmıştır.

```
def train_mlp_standard(self, X_train, y_train, ...):  
    model.compile(optimizer='adam', loss='mse')  
  
    early_stop = callbacks.EarlyStopping(  
        monitor='val_loss', patience=30, restore_best_weights=True  
    )  
  
    history = model.fit(  
        X_train, y_train,  
        validation_data=(X_val, y_val),  
        epochs=200,  
        callbacks=[early_stop, reduce_lr]  
    )  
    return model, history
```


6. Ensemble Yöntemleri

Basit Ortalama

RF, XGBoost ve MLP tahminlerinin aritmetik ortalaması alınır.

$$\text{pred} = (\text{RF} + \text{XGB} + \text{MLP}) / 3$$

Ağırlıklı Ortalama

Modellerin doğrulama performansına göre ağırlıklandırılması.

- MLP: %50
- XGBoost: %25
- RF: %25

Stacking

Temel model tahminlerini girdi olarak kullanan bir Meta-Model (Ridge Regression) eğitilir.

(Bu projede MLP'nin tek başına başarısı nedeniyle ensemble sınırlı katkı sağladı)

7.1 Model Karşılaştırma Sonuçları

Tüm modellerin test seti üzerindeki performans özeti:

Model	Ortalama R ²	RMSE (%)	MAE (%)
MLP Standard (En İyi)	0.863	5.87	4.03
Ağırlıklı Ensemble	0.797	6.94	4.75
Basit Ortalama	0.746	7.69	5.16
XGBoost	0.603	9.50	6.10
Random Forest	0.571	9.83	6.25

7.2 MLP Standard: Bileşen Bazlı Doğruluk

MLP modeli, özellikle Aromatikler ve Furanlar gibi ana bileşenlerde çok yüksek doğruluk göstermiştir.

Bileşen	R ² Skoru	MAE (%)
Aromatikler	0.942	6.35
Furanlar	0.897	1.05
Asitler	0.877	4.80
Alkoller	0.853	3.41
Aldehitler-Ketonlar	0.849	3.29

7.3 Özellik Önem Dereceleri

Singazın Gücü

Analizler, singaz kompozisyonunun tahmin gücünün %88.4'ünü oluşturduğunu göstermektedir.

Sıcaklık ve basınç gibi proses koşullarının etkisi (%4.6) daha sınırlıdır, çünkü termodinamik denge zaten bu koşulların sonucudur.

En Önemli Faktörler

1. **CH₄ (%27.1)**: C/H oranının en güçlü göstergesi.
2. **CO₂ (%26.0)**: Oksijen içeriğini yansıtır.
3. **H₂O (%20.5)**: Buhar reaktivitesi.
4. **H₂ (%11.9)**

8. Kullanım Örneği (Tahmin)

Eğitilmiş modelin (`mlp\_standard.h5`) yeni bir veri noktası üzerinde tahmin yapmak için kullanımı.

```
# 1. Modeli yükle
model = keras.models.load_model('mlp_standard.h5')

# 2. Yeni veriyi hazırla (Örn: 750°C Reformer)
syngas = pd.DataFrame({
    'Reformer_Temperature_C': [750],
    'H2_molpercent': [32.97],
    'CO2_molpercent': [15.06],
    ...
})

# 3. Normalize et ve tahmin yap
X_scaled = scaler_X.transform(syngas)
prediction = model.predict(X_scaled)
print(f"Aromatikler: {prediction[0][0]:.1f}%")
# Çıktı: Aromatikler: 42.3%
```


9. Bağımlılıklar ve Kurulum

Temel Kütüphaneler

- `pandas ≥ 1.5.0`
- `numpy ≥ 1.23.0`
- `scikit-learn ≥ 1.2.0`

ML & DL

- `tensorflow ≥ 2.12.0`
- `keras ≥ 2.12.0`
- `xgboost ≥ 1.7.0`

Özel Araçlar

- `cantera ≥ 2.6.0` (Simülasyon)
- `pyodbc` (Veritabanı)
- `matplotlib/seaborn`

```
pip install -r requirements.txt
```


10. Sonuç ve Katkılar

Bilimsel Katkılar

- **İlk Ters Model:** Literatürdeki ilk ters biyo-yağ reforming tahmin modeli.
- **DL Üstünlüğü:** MLP, klasik yöntemlerden %51 daha iyi performans gösterdi.
- **Özellik Önemi:** CH₄ ve CO₂'nin belirleyici rolü kanıtlandı.

Pratik Değer

- **Hızlı Tarama:** Cantera'dan 1000 kat daha hızlı tahmin.
- **Yüksek Doğruluk:** %4.03 Ortalama Hata (MAE).
- **Sanayi Uyumu:** Gerçek zamanlı süreç izleme için uygun.

Teşekkürler

Sorularınız?

Orhun Uzdiyem | Aralık 2024